

Project

Learn OOP The Hard Way 1

Modeling and Implementing a Machine Learning Library

Important notes

This is an OOP project with a machine learning flavor. Therefore, particular attention will be given to the design/modeling and elegance of your solution, which has to leverage all the power of OOP principles and Python features (decorators, properties, static/class methods, inheritance, etc.) seen during the course (and beyond). Note that to reach such a solution (cleanness of the design and elegance of the solution), it requires many iterations, so do not optimize for cleanness and elegance immediately.

Along the development iterations, you will make design decisions (trading between advantages and disadvantages) about your solution. We ask you to keep track of all these design choices and explain them in your report. It is also advised to keep track of the successive versions of your class (and object) diagram in a folder `/docs/diagrams/` with a naming convention like `class-diagram-v*.png` (where `*` corresponds to the version number starting from 0).

Regarding the report, it should provide at least the following elements, but you are free to be creative:

1. A class diagram that provides a high-level overview of your solution, along with a section that provides the general principles (or philosophy) of your approach to modeling the library;
2. A series of subsections focusing on different parts of your model, with a detailed description of these parts accompanied by **some portions** of code (of **particular interest**). You can also talk about the history (the successive iterations) that led you to that specific modeling;
3. A section dedicated to the evaluation of your library, including the description of the different tests you performed on the library and the results that you obtained. You are able to perform comparisons with the performance that you would get with the original Pytorch library, so do not hesitate to draw comparative charts regarding computation time, convergence of the results, etc.;
4. A section dedicated to the limits of your model and implementation: what didn't work? Why (e.g., which design choices led you to that)? And what can you do about it if you have a chance to continue working on this project?

Don't use ChatGPT to generate your report (nor to generate the entire code). Any abuse will not be tolerated. Be concise in your writing: we do not ask you to write a novel, but simply what you did.

All developments will be done in the GitHub organization dedicated to this course. Make regular commits to your respective repositories. Recall that a commit should correspond to implementing/solving/fixing a very specific aspect in your code or model. Recall also that it is essential to provide clear messages for your commits. This way, one can retrieve commits related to a particular aspect and ease version control (e.g., reverting changes).

The repositories will be private. Do not share your models nor implementation with other groups. Chances are that you will end up with similar code. Again, any abuse will not be tolerated.

The project must be completed by groups of two to three students.

Be aware that it is forbidden to use scikit-learn within your library. We ask you to build something similar to scikit-learn. Therefore, the following kind of code, for example, is **not legit**:

```
class KMeans:
    def __init__(self, k, ...):
        import sklearn
        self.kmeans = sklearn.KMeans(k)
        self.k = k
        [...]
```

Functional Description

Models (*upsacalay_ml.models*)

Estimator

An estimator serves as the fundamental component of the machine learning models within our library. An estimator can be seen as a function $f: X \rightarrow Y$ that is used to approximate an unknown function or phenomenon from the real world (cf. machine learning course). It takes as input a matrix (table or training set) X , containing a set of observations as rows, and predicts a set of labels Y , e.g., for each observation, the cluster that the estimator assigns it to.

Estimator

Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DPF	Age
6	148	72	35	155.55	33.6	0.627	50
1	85	66	29	155.55	26.6	0.351	31
8	183	64	29.15	155.55	23.3	0.672	32
1	89	66	23	94	28.1	0.167	21
0	137	40	35	168	43.1	2.288	33
5	116	74	29.15	155.55	25.6	0.201	30
3	78	50	32	88	31.0	0.248	26
10	115	72.41	29.15	155.55	35.3	0.134	29
...	...	...	...	...	...	...	...



Training set

Figure 1:

- Input matrix (table or training set) denoted X . It contains a set of observations as rows, each described by a set of features as columns.

- Fitting an *estimator* to a dataset has the effect of training the estimator on the data. In the case of k-means, for example, fitting a k-means to a dataset has the effect of running the k-means algorithm on the dataset starting from a random initialization of the centroids until stable clusters of data are found.
- The result of this procedure is a trained estimator.

Models to implement

We distinguish two types of models (or algorithms): (1) *supervised* and (2) *unsupervised*. The former models require an explicit set of target labels, denoted Y , against which the model's predictions will be compared to evaluate its performance. The latter models, on the contrary, do not need the provision of a set of labels. Instead, they predict the membership of the observations to clusters.

You have to implement the following models, which were covered in the machine learning course. You are free to implement additional models, but the following ones are mandatory.

1. Supervised models:

- Decision tree classifier

2. Unsupervised models:

- K-means
- Agglomerative clustering
- Spectral clustering

All these models share some standard methods, such as `fit`, `fit_transform`, etc. These methods run the corresponding learning algorithm on the provided set of observations and labels.

Datasets (*upsaclay_ml.datasets*)

Your library must support handling the input matrices.

- The library must support loading any CSV file, either by providing the file path or a URL from which it can be downloaded.
- The library must also provide native, rapid access to widely used datasets, such as MNIST, and to omics and health-related datasets, such as TCGA. You are free to add support in your library for any suitable dataset you may think of. Beware of spending too much time adding support to many datasets while sacrificing other aspects of the library. While it will be appreciated, this will not compensate for potential weaknesses of other aspects of the library.
- The library must provide support for a dataloader that loads data in mini-batches.

For example:

```
dataset = MNIST() 1
print(dataset[3]) 2
print(dataset) 3

dataloader = DataLoader(dataset, batchsize=32, transform=None) 4
print(len(dataloader)) 5
for batch in dataloader: 6
    print(batch.shape)
```

In particular, based on this example:

- ❶ Native and easy access to the MNIST dataset
- ❷ Select an element from the dataset via indexing
- ❸ Redefine the behavior of `print()` so that one can visualize a summary (or statistics) of the dataset
- ❹ Create a dataloader from any dataset
- ❺ Getting the length of the dataloader
- ❻ Iterating over a dataloader should return a sequence of batches of examples.

Transformers (`upsacLay_ml.transformers`)

Before fitting any model to a dataset, we usually apply preprocessing transformations to make the data more suitable for training, for example, removing outliers, normalizing values, standardizing features, etc. Your library must provide a minimal set of preprocessing transformations that can be applied to the dataset.

Transformer (also an Estimator)

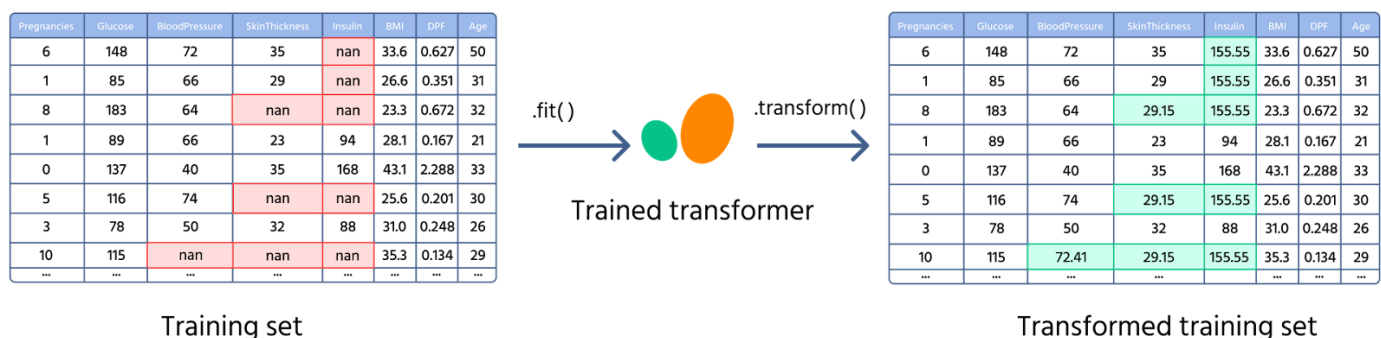


Figure 2:

- Fitting a *transformer* is similar to fitting an estimator. For example, fitting a `StandardScaler` on a dataset has the effect of computing the mean of each feature.
- Beware that fitting alone does not modify the dataset. Applying the transformation—specifically, standard scaling—in this case requires explicitly calling `transform`. In other words, we need the library to offer the ability to (1) fit a transformation to a given dataset and (2) apply the transformation to a given dataset.

Note: models (cf. Section Models above) share the same methods with transformers. This way, transformers and models can be chained in the form of a pipeline (see Section Pipeline below).

The following are some preprocessing transformers that your library must provide:

Standard scaler

Standardizes features by removing the mean and scaling to unit variance.

$$x' = \frac{x - \mu}{\sigma}$$

Where μ is the mean and σ is the standard deviation of each feature.

Min-Max scaler

Scales each feature to a fixed range, typically [0,1].

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Binarizer (threshold-based)

Converts numerical values into binary (0/1) values based on a threshold.

$$x' = \begin{cases} 1 & \text{if } x > \text{threshold} \\ 0 & \text{otherwise} \end{cases}$$

One-hot encoder

Converts categorical variables into a set of binary (0/1) features.

Each category becomes a new column, with a 1 indicating the presence of that category and a 0 otherwise.

Example:

Color = "red", "green", "blue" $\Rightarrow$ [1, 0, 0], [0, 1, 0], [0, 0, 1]

Pipeline (*upsacalay_ml.pipeline*)

Your library must provide a way to chain multiple transformations and ML models in a seamless, convenient manner, as depicted, for example, in Figure 3. In this example, we can see a pipeline containing three transformations and a final machine learning model, stacked one on top of the other.

Fitting a pipeline involves applying the various transformations and the model to the provided dataset: the pipeline first applies the initial transformation to the original dataset; then, each subsequent transformation is applied to the result of the previous one; finally, the model is applied to the result of the last transformation.

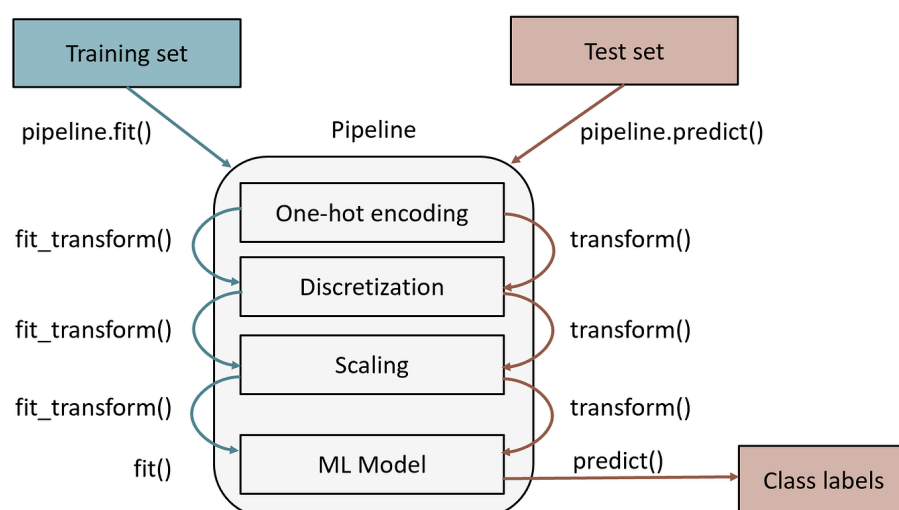


Figure 3:

- A pipeline containing: One-hot encoding; Discretization; and Scaling transformers.
- A final ML Model is added at the end of the pipeline
- `pipeline.fit()`
 - Calling the method `fit()` on this pipeline (i.e., `pipeline.fit()`) using the dataset “Training Set” (in blue) has the effect of calling the methods `fit_transform()` of each transformer in the pipeline in a cascade fashion
 - Finally, the `fit()` method of the ML Model is called, which has the effect of fitting the ML Model on the transformed dataset. This will result in a fitted estimator.
- `pipeline.predict()`
 - When testing the model, you can call the method `predict()` of the pipeline (i.e., `pipeline.predict()`) on the dataset “Test set” (in brown)
 - This has the effect of calling the `transform()` method of each transformer in the pipeline in a cascade fashion
 - Finally, the `predict()` method of the ML Model is called. This will return an array of predictions.

Utilities (`upsacalay_ml.model_selection` and `upsacalay_ml.metrics`)

Model selection

Train-test split: splits a dataset into separate training and testing subsets to evaluate a model's generalization performance.

- It can take a parameter that specifies the proportion (float) or count (int) of samples in the test set.
- It can also take a parameter specifying whether to shuffle the data before splitting.

Metrics

- supervised: accuracy score, mean squared error, etc.
 - Accuracy score: Measures how often the classifier correctly predicts labels.
 - Mean Squared Error: Measures the average squared difference between predicted and actual values.
- Unsupervised: silhouette score, etc.
 - Silhouette score: Evaluate the quality of clustering by comparing how similar samples are to their own cluster versus other clusters (cf. course).

Implementation

Your implementation should make full use of the concepts covered in the OOP course, for example, dunder methods, class decorators, and, more generally, object-oriented principles.

Dunder methods

Each major class (e.g., `BaseEstimator`, `Pipeline`, and models) must implement at least a set of dunder methods. Examples:

- `__repr__` / `__str__`: Pretty-print with hyperparameters.
- `__eq__`: Compare models by class + hyperparameters.
- `__len__`: For `Pipeline`, return the number of steps.
- `__getitem__` / `__setitem__`: Access pipeline steps by index or name.

- `__iter__`: Iterate through pipeline steps.
- `__call__`: Shortcut for predict.

Class decorators

Here are some examples of class decorators that can be implemented, either to be used by the developer (externally) or within your library (internally):

- Check hyperparameter values at class initialization.
- register model
- timed fit
- frozen model
- log calls to methods and arguments
- cache
- track history
- serialize model

These examples are kept vague deliberately to let you propose suitable, useful, and original class decorators.

Usage

You have to make your library user-friendly and easy to use. Here are some examples of how a potential developer can use your library.

Supervised learning

This is an example of how to use your library to apply a decision tree on the Iris dataset.

```
from upsacalay_ml.models.tree import DecisionTreeClassifier
from upsacalay_ml.datasets import Iris
from upsacalay_ml.model_selection import train_test_split
from upsacalay_ml.metrics import accuracy_score

X, y = Iris()
X_train, X_test, y_train, y_test = train_test_split(X, y)

clf = DecisionTreeClassifier(max_depth=3)
clf.fit(X_train, y_train)
print("Accuracy:", accuracy_score(y_test, clf(X_test)))
```

Unsupervised learning

This is an example of how to use your library to apply a k-means on the blobs (toy) dataset.

```
from upsacalay_ml.models.cluster import KMeans
from upsacalay_ml.datasets import Blobs
from upsacalay_ml.metrics import silhouette_score

X = Blobs(n_samples=200, centers=3)

kmeans = KMeans(n_clusters=3)
```

```
labels = kmeans.fit_predict(X)

print("Silhouette:", silhouette_score(X, labels))
```

Pipeline

This is an example of how to use your library to stack a set of transformations and a linear regression in the form of a pipeline and apply it to a toy dataset.

```
from upsaclay_ml.pipeline import Pipeline
from upsaclay_ml.models.linear_model import LinearRegression
from upsaclay_ml.transformers import StandardScaler
from upsaclay_ml.datasets import Dataset
from upsaclay_ml.metrics import mean_squared_error

X, y = Dataset("./path/to/linear.csv")
pipe = Pipeline([
    ("scaler", StandardScaler()), ❶
    ("model", LinearRegression())
])

pipe.fit(X, y)
y_pred = pipe(X) ❷
print("MSE:", mean_squared_error(y, y_pred))
```

- ❶ A pipeline can (among other options) be defined via a list of tuples (name, transformer/estimator)
- ❷ A fitted pipeline can be called directly on the data, and in this case, it will ultimately call the predict method of the ML model, here LinearRegression.

Continuous integration/functional test suite and evaluation

Your library has to allow users to build and train their models. A set of functional tests will be run on your implementation to determine whether the different functionalities are implemented and whether they are implemented correctly. For this, a continuous integration infrastructure has been set up within GitHub to test your implementation continuously. As shown in Figure 4, the test suite summary indicates which functional

The screenshot shows a GitHub Actions workflow run for 'Run Tests #16'. The workflow is successful, with a total duration of 27s. A summary box highlights 'Functional Test Results' showing two failed test cases: 'test_kmeans_clusters' and 'test_kmeans_clusters', both marked as FAILED.

Test Case	Result
test_kmeans_clusters	❌ FAILED
test_kmeans_clusters	❌ FAILED

tests have passed or failed (red rectangle). These will help you continuously fix your implementation and determine which functionalities were implemented correctly and which need further refinements.

Figure 4:

- The GitHub interface provides you with an “Actions” tab
- This figure shows details of a specific run of the tests
- The summary results of the tests are highlighted within the red rectangle.

These functional tests are not visible to you. However, you can run the tests yourself by going to Actions->”Run Tests”->Run workflow. You have to wait until the workflow has finished. You will get the summary of the test results in the panel. See Figure 5.

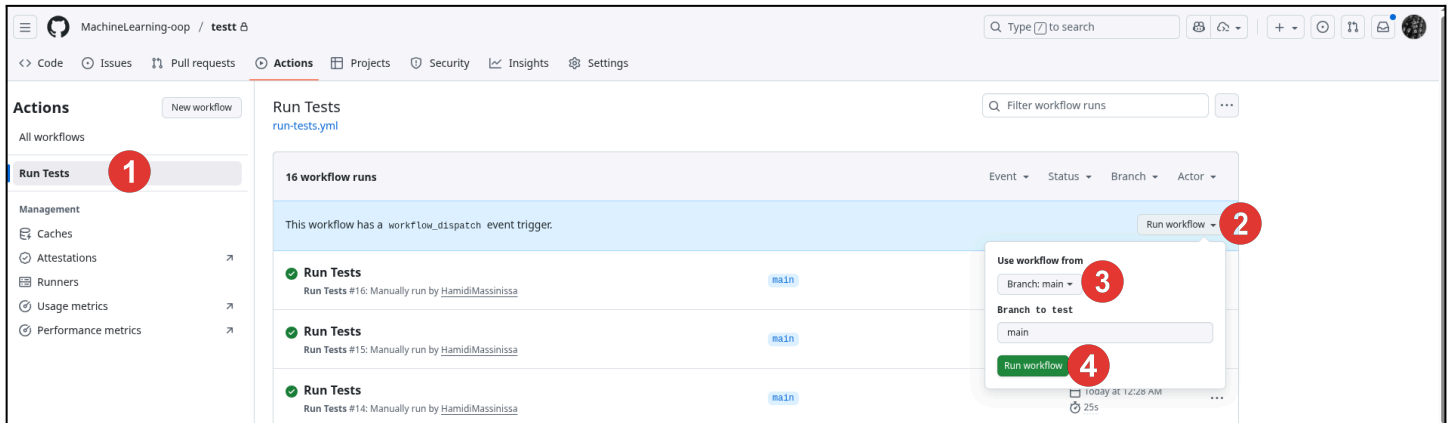


Figure 5:

- **1** Click on the workflow “Run Tests”
- **2** Click on “Run workflow”
- **3** Choose the branch containing the code to test. Keep it as “main” if you have only this branch
- **4** Click on Run workflow

Note that the tests should be run **parsimoniously**. The tests take time to run, and GitHub limits the total time allocated to the GitHub organization. A per-group overview is available to ensure proper usage. The tests are here to help you. Thanks for your collaboration.